RAG 全链路技术详解

原创 天猫品牌行业架构 大淘宝技术 2026年5月18日 02:36 浙江 1190人

本文为RAG技术实战指南，聚焦Agent开发中的核心挑战：知识库构建不规范、检索召回不准、缺乏量化评测。全文覆盖RAG全链路——从文档加载（多格式解析+元数据提取）、智能切分（规则/语义/结构化方法，含Meta-Chunking原理）、索引构建（embedding模型选型与向量生成详解）、检索优化（Query改写、HyDE/Doc2Query、标签过滤、重排序）、生成调优（Prompt设计、参数控制、SFT微调），到进阶Graph RAG（多跳推理与全局摘要），最后落地Ragas自动化评估体系（Context Precision/Recall、Faithfulness、Answer Relevancy等指标及测试集生成）。强调“可测、可调、可信赖”的RAG工程化实践。

01 写在前面

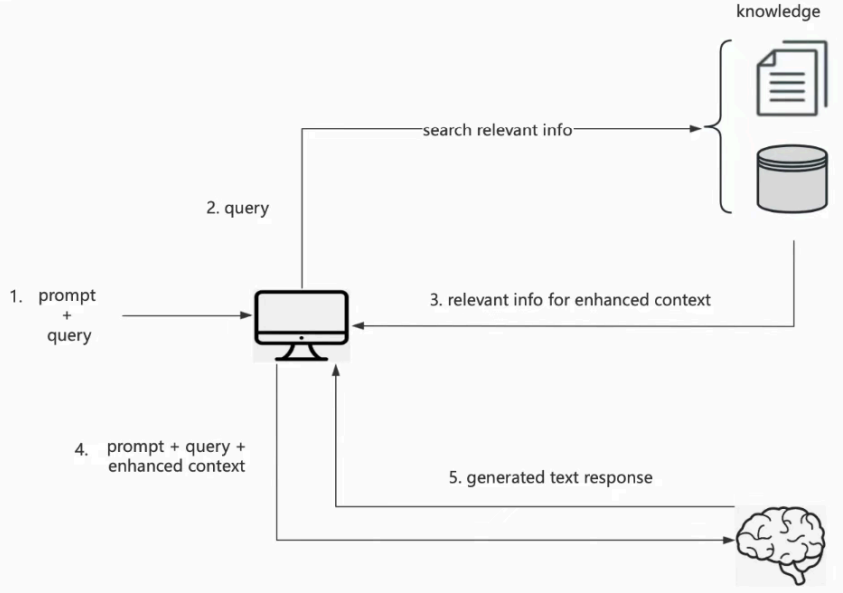
在Agent的开发过程中，RAG技术的应用水平直接决定了Agent的业务上限。在跟进品牌行业AI运营战役时，架构组发现部分团队在RAG落地中面临共性挑战：知识库构建缺乏标准、检索召回精度达不到预期，以及缺乏量化评测体系。因此，我们梳理了这篇技术指南，期望从实战角度拆解RAG的全链路核心能力：从底层原理出发，深入探讨如何构建索引、如何通过技术手段优化召回，以及如何建立科学的评测闭环。希望通过这篇文章，大家能看清RAG背后的技术细节。不仅能把Agent做出来，更能通过调优和评测，构建出低幻觉、具备业务确定性的Agent应用。

02 为什么需要RAG?

大型语言模型（LLMs）已经取得了显著的成就，尽管它们仍然面临着很大的局限性，尤其是在特定领域或知识密集型任务中，特别是在处理超出其训练数据或需要当前信息的查询时，会产生 "幻觉"。为了克服这些挑战，检索增强生成（RAG）通过语义相似性计算从外部知识库中检索相关文档块，从而增强了 LLM。通过引用外部知识，RAG 可有效减少生成与事实不符内容的问题。

03 RAG的工作原理

RAG，全称Retrieval Augmented Generation（检索增强生成），其工作原理为：当用户提问时，系统先从海量知识库中检索出与问题相关的实时或私有信息，并将这些信息作为参考背景与问题一起提供给大模型，让模型像做**开卷考试**一样，基于提供的可靠证据生成更准确、实时且可溯源的回答。



04 RAG核心技术点

文档加载

- 文档加载是RAG的“入口”，它的质量直接决定了后续检索的准确性。简单来说，**它的工作就是将各种格式的非结构化数据转化为系统可处理的统一文本格式**。
- 文档加载的主要工作如下：
  - 多格式适配： 兼容各种文件格式，如 PDF、Word、Markdown、HTML、JSON、TXT 甚至 Excel。
  - 内容提取： 从原始文件中剥离出“净文本”。
    - 如果是 PDF，需要解析文字层；如果是扫描件，则需要 OCR。
    - 如果是 HTML，需要剔除脚本、样式表，只保留正文。
  - 元数据（Metadata）提取： 在提取文本的同时，记录文档的属性，如：文件名、页码、标题、作者、创建时间、章节层级等。
  - 初步清洗： 去除文档中的冗余信息，如乱码、过多的空格、特殊的非打印字符等。

文档切分

- 文档切分的主要方式如下：

分块方法	方法描述	优点	缺点	适用场景
按token数分块	预先设定一个固定的字符数或 Token 数（如 512 个 Token），将文档机械地拆分为等长的块，**通常会设置一定的重叠度 (Overlap) 以减少连接处的信息丢失。**	实现极其简单，计算开销最小，不依赖任何模型。	完全忽略语义结构，极易在句子或段落中间断开，导致语义碎片化。	对性能要求极高、处理超大规模无结构文本，且对检索精度要求不高的**初步实验。**
递归字符切块	通过一组层级分隔符（如 ["\n\n", "\n", " ", ""]）递归地拆分文本。**它首先尝试按段落拆分，如果段落太大，再按行拆分，以此类推，直到切块大小符合要求。**	能尽可能保持段落和句子的完整性，比固定切块更贴合人类阅读习惯。	依然是一种规则驱动的硬切，**对于极其复杂的文档结构，仍可能丢失深层语境。**	通用的 PDF、Word 文档、普通文章，是大多数 RAG 项目的默认首选。
基于结构的切块	针对特定格式的文档，利用其自身的语法标记进行切块。 解析 Markdown、HTML、JSON的语法树。例如，Markdown 按标题层级（H1, H2, H3）切块。	完美保留文档的逻辑层级，检索到的内容通常具有极高的自解释性。	高度依赖输入文档的格式规范性。如果 Markdown 格式混乱，效果会大打折扣。	技术文档、API 文档、代码库、结构化的 Web 页面
按语义分块	基于文本背后的“意思”而非物理符号来决定切分点。 先将文档拆分为句子，利用 Embedding 模型将句子向量化。计算相邻句子间的余弦相似度，当相似度低于某个阈值（说明话题转折了）时，进行切块。	保证了每个块内的语义高度统一，极大提升了检索的相关性。	由于需要对所有句子进行向量化计算，计算成本高，处理速度比规则切块慢得多。	长篇报告、叙事性文学、话题转换频繁的深度内容。

- 按语义切块的实现（Meta-Chunking）

- Meta-Chunking: Learning Text Segmentation and Semantic Completion via Logical Perception：该论文给出了一个基于逻辑和语义的chunking的方法，有完整的效果评测，可参考。

  原理

- 计算文章中各句子（sentence）的PPL（困惑度），以此判断哪些句子应该被划分在一个文档块（trunk）中。

> PPL（Perplexity，困惑度）源于信息论，是衡量语言模型预测序列好坏的核心指标。在该论文中，PPL 被巧妙地用作一种感知语义边界和逻辑完整性的度量工具。

> **直观理解：** PPL 反映了语言模型在看到一段文本时有多“困惑”。低 PPL： 模型觉得这段话很通顺、逻辑连贯、符合语言规律；高 PPL： 模型觉得这段话很突兀、逻辑断层或不符合语

> 境。

实现步骤

- 对文档进行切分：
  - 将原始长文档按照标点符号切分成一个个句子（Sentences）。切分好的句子表示为，$(x_1, x_2, \ldots, x_n)$。
  - 使用一个轻量化的大模型（论文中常用 Qwen2）“计算”每个句子与他前面句子的PPL（困惑度）。其中，K表示x_i中的token数。t^i_k表示x_i中的第k个token，$t_{<i}$表示所有位于x_i之前的token。通过这种方式，得到一条随文本流动的 PPL 曲线。

$$\mathrm{PPL}_M(x_i) = \frac{\sum_{k=1}^{K} \mathrm{PPL}_M(t_k^i | t_{<k}^i, t_{<i})}{K}$$

  - **寻找 PPL 曲线上的局部极大值**（Local Maximum）。当模型处理到某一句时，如果 PPL 突然大幅升高，说明这一句与前面的内容在逻辑上“脱节”了。这个 PPL 飙升的点就被标记为潜在的切分边界。
- 对每个切块（trunk）进行**语义补全**：解决因文本分块导致上下文信息丢失而引发的语义鸿沟问题。
  - 全局增强的文本块重写（Globally Augmented Text Chunk Rewriting）：**for生成质量**
    - 预处理（可选）：针对 LLM 难以一次性完全处理的极长文档，采用embedding模型对每个trunk进行向量化，识别出与当前trunk相近的trunk。
    - 推理是否丢失信息： 利用 LLM 并结合预处理阶段识别出的潜在相关信息，对每个文本块判断是否丢失信息。核心任务是明确识别当前文本块中**缺失的前提、背景信息、相关事实或结论性陈述**。LLL 应全面列出信息缺失的领域，并具体说明需要补充的信息。
    - 丢失信息提炼：对上一阶段检测到的潜在丢失信息进行打分和筛选。我们旨在防止引入无关或错误的补充内容，从而确保增强过程的精确性。
    - 丢失信息补全：基于前一阶段确认的必要信息，引导 LLM 生成这些信息片段并与当前文本块进行整合。目标是生成一个在上下文中无缝衔接、语义自然且能有效实现跨块信息融合的新文本块。
  - 摘要生成（Context-Aware Summary Generation）：每个文本块生成一个富含全局信息的简明摘要，从而进一步增强该文本块的语境感知能力。**for检索**
    - 全局补充：模型利用全局信息为目标文本块生成一份补充摘要。这一过程旨在补偿由于切分（Segmentation）而可能缺失的话语背景（Discourse Background）和外部关联信息。
    - 局部融合：针对文本块本身的内容，模型会独立生成一个局部摘要（Local Summary），以概括其核心观点。随后，将前述的“补充摘要”与“局部摘要”进行融

合与精炼，最终形成一句话的增强摘要（Enhanced Summary Sentence）。该句子能够从全局视角清晰地表达该文本块的内容。

索引构建（embedding）

- 基本原理

索引构建是为了将一段文档根据其语义去生成向量。与传统embedding通过“预测”任务把文档转换成向量不同，现代embedding模型（千问 text-embedding-v4等）基于Transformer架构，将文档通过深度学习模型映射到连续的向量空间。下面以“苹果手机很好用”为例来详细看下映射过程：

- **分词与编号（Tokenization）：**
  - 将“苹果手机很好用”切分为：[苹果，手机，很好用]。
  - 在头尾增加特殊字符：[CLS]：放在句首，代表“分类/全句汇总”信息；[SEP]：放在句尾，代表句子结束。
  - 将token转换为id：根据embedding模型中预设好的词表，将token转换为数字索引。[CLS，苹果，手机，很，好用，SEP] -> [101，512，678，321，102]。
- **映射初始向量：**在进入复杂的计算层之前，将数字索引变成一个基础向量。
  - 查表： 查阅embedding模型中预定义的表格，每个ID拿到一个例如1536维的初始向量。
  - 转换：模型有三个权重矩阵（W^Q，W^K，W^V）。这三个矩阵是模型训练好后固定不变的参数。**每个token都会通过数学变换变成三个角色（向量）**，我们称之为 Q、K、V。其中，**Query（Q）**——“提问者”：我想找什么样的主体？**Key（K）**——“被索引者”：我能提供什么样的特征？**Value（V）**——“内容本身”：我实际代表的含义。

$$
\begin{aligned}
Q &= \text{初始向量} \times W^Q \\
K &= \text{初始向量} \times W^K \\
V &= \text{初始向量} \times W^V
\end{aligned}
$$

  - 注入位置信息（Positional Encoding）：给每个向量额外加上一组代表位置的数字。这样模型就知道“苹果”在第1位，“很好用”在第3位。其中，X是与位置相关的矩阵。

$$
\begin{aligned}
Q &= XW^Q \\
K &= XW^K \\
V &= XW^V
\end{aligned}
$$

- **自注意机制（Self-Attention）：**
  - 计算每个token与其他token的相关性：
    - “苹果”的 Q × “苹果”的 K = 分数很高（自关联）
    - “苹果”的 Q × “苹果”的 K = 分数很高（语义强相关）

        - “苹果”的 Q × “很好用”的 K = 分数一般
    - 分配权重（归一化）：刚才计算的分数转化成百分比
        - “苹果”的 Q × “苹果”的 K = 100%
        - “苹果”的 Q × “苹果”的 K = 50%
        - “苹果”的 Q × “很好用”的 K = 10%
    - 重新计算每个token的向量：模型把所有词的 V（内容本身） 按照刚才的百分比“混合”在一起：
        - “苹果”的新向量 = 100% * “苹果”的V + 50% * “手机”的V + 10% * “很好用”的V。
        - “苹果”的向量会吸收“手机”的特征。“苹果”向量里的“水果/红色”特征被抑制，而“电子产品/科技”特征被放大。
    - 逐层深化：以上过程通常会重复 12 层或 24 层。
        - 随着层数增加，每个 Token 的向量都不再是原来的意思，而是包含了整句话信息的“超级词向量”。比如“好用”的向量里现在也包含了“手机”的信息。
- **优池处理（Pooling）**：将每个token的向量合并成一个向量。
    - CLS 方式： 直接取第一个[CLS]（句首）位置对应的向量。模型认为这个位置的向量已经吸收了全句的精华。
    - Mean Pooling（平均池化）： 把 5 个向量相加取平均值（这是目前 BGE 等 RAG 模型最常用的方式，因为它能最平稳地代表全句含义）。
    - Max Pooling（最大池化）： 提取 5 个向量中每一维度的最大值，保留最强烈的语义特征。
- **向量归一化（L2 Normalization）**：将池化后生成的向量进行归一化，即使其模长（长度）等于 1。归一化后，两个向量的点积（Dot Product）就直接等于它们的余弦相似度。这大大加快了在向量数据库中搜索的速度。

- 常用的embedding模型

厂商	模型名称	最大输入token	默认维度	MIRACL榜单得分	千token价格（RMB）	适用场景
OpenAI	text-embedding-3-small	8191	1536	0.44	0.00014	便宜、在英语语言下效果较好
	text-embedding-3-large	8191	3072	0.549	0.00091	复杂维度需求，可以在3072 维下捕捉极其细微的语义差异，能捕捉到极其细微的语义差别
阿里云-千问	text-embedding-v3	8191	1024	0.702（曾排名全球第一）	0.0005	• 专业级中文 RAG 系统；超长文档处理；国内主流应用 • 备注：v4已发布适用版，AiStudio可选择
阿里云-OpenSearch	ops-text-embedding-002	8192	1024	——	0.0005	电商/生活服务搜索、大规模商用搜索引擎、推荐系统
谷歌	textembedding-gecko	3072	768	0.5~0.65	0.0011 ~ 0.0015（信息不一定准确）	对延迟（Latency）极度敏感的业务、海外多语言应用（尤其是在拉丁语系国家，Gecko 的泛化能力非常强）
北京智源人工智能研究院（BAAI）	bge-large-zh	512	1024	0.65 - 0.73	开源模型，需自己部署	可私有部署、需要微调、短文本匹配优化（擅长处理 FAQ匹配，适合智能客服的问答对检索）

备注：部分数据来自大模型，仅供参考。

检索

检索阶段会召回与问题最相关的文本段。通过embedding模型对问题进行文本向量化，并与向量数据库的段落进行语义相似度的比较，找出最相关的段落。为大模型提供高质量、强相关的参考证据，直接决定了生成答案的准确性、相关性以及消除幻觉的上限。

- Query改写

Query优化

- 核心目标：对用户原始查询进行优化，纠正原始问题中的偏差、模糊或不完整信息，以提高检索效果。
- 解决的问题：
  - 指代消解（Coreference Resolution）： **处理多轮对话**。例如用户问“它的价格是多少？”，改写器会根据上下文将其改写为“华为 Mate 60 Pro 的价格是多少？”。
  - 纠错与去噪： 修正错别字，剔除无意义的口语助词（如“那个...我想问一下...”）。
  - 术语对齐： 将口语化的表达转化为专业术语。例如将“肚子疼怎么办”改写为“腹痛的治疗建议”。
  - 结构转换： 将复杂的长难句拆解或简化，提取出核心语义特征。

多查询生成

- 将单一查询转化为多查询，通过“广撒网”和“多维视角”来提升检索质量的技术。这种方法通常利用大模型针对用户的原始问题，从不同角度生成 3-5 个意思相近但表达不同的问题，然后分别进行检索。**核心是提升召回率。**
- 例子：
  - 用户问“如何提高睡眠质量？”。扩展成：Query 1（侧重生活习惯）：“睡前哪些习惯有助于入睡？”；Query 2（侧重环境因素）：“什么样的卧室环境适合深度睡眠？”Query 3（侧重医学建议）：“改善失眠的科学方法有哪些？”
  - 效果： 相当于在向量空间里投下了多枚“探测器”。只要其中一个探测器接近目标，就能抓回正确答案。防止因为用户提问太片面而漏掉关键资料。

HyDE：用假设文档增强检索

- HyDE

  （Hypothetical Document Embeddings）： 不是直接用“问题”去搜“答案”，而是先让 AI 瞎编一个“假答案”，再用这个“假答案”去搜“真内容”。**本质是将“问题-文档匹配”转换成了“文档-文档匹配”**。
- 解决的问题：在传统的向量检索中，我们面临一个“非对称检索“的问题。用户的问题： 通常很短，带有疑问语气（例如：“为什么天空是蓝色的？”）。知识库里的文档： 通常很长，是陈述性的（例如：“瑞利散射是指……导致波长较短的蓝色光散射更强……”）。在向量空间里，“问题”和“答案”的语义特征其实并不完全对等。有时候，系统很难仅凭一个短小的问题就精准捕捉到长篇大论的答案。
- HyDE 的工作流程：
  - 接收提问： 用户输入一个问题（Query）。
  - 生成虚构文档（Hypothetical Document）：
    - 系统调用大模型（LLM），给它一个指令：“请写一段话来回答这个问题”。
    - 注意：此时模型并不去查资料，它只是凭自己的“直觉”或内部知识编一个答案。即便这个答案里包含错误的信息也没关系。
  - 向量化（Embedding）： 将这个虚构的答案转化成向量。
  - 匹配检索： 用这个“虚构答案的向量”去数据库里找最相似的真实文档。
- 例子：
  - 用户问：为什么面团放一会儿会变大？
  - 知识库中的内容包含的是：“酵母发酵”、“二氧化碳释放”、“面筋网络”或“厌氧呼吸”等专业原理。
  - LLM写一个伪答案：面团之所以会体积膨胀，是因为其中的酵母菌在适宜的温度下通过发酵作用分解糖分。这个过程产生了大量的二氧化碳（CO2）气体，气体被面粉中的面筋蛋白结构包裹，形成无数微小气泡。随着气泡增多和膨胀，整体面团的体积就会增大。
  - 根据伪答案去知识库做向量检索。知识库中有一篇深奥的论文，题目叫《真菌代谢对谷物蛋白弹性的影响》。论文中包含“二氧化碳”、“酵母代谢”、“面筋网络弹性”等词汇。因为

伪答案里也有这些词，两者的向量在空间中高度重合。

- 模型总结最终回答：根据生物学原理（参考《真菌代谢...》论文），面团变大是因为酵母发酵产生了二氧化碳。这些气体被面筋蛋白锁定，导致体积扩张……

- 反向HyDE（Doc2Query）

- 针对chunk，生成这块chunk可能的question，然后针对这些question进行索引构建，关联到具体的chunk内容；**反向HyDE相比HyDE的优势是可以离线处理，不影响实时调用的rt；**
- 例子：
  - 数据准备：

> 原始文本：“根据公司差旅管理办法第四条，员工在非一线城市出差时，每日住宿补贴上限为 350 元人民币。申请人需在差旅结束后 15 个工作日内，通过 ERP 系统提交正规增值税专用发票进行报销，逾期将不予受理。”

    - 让大语言模型针对这段话生成 3-5 个“虚构问题”：①出差去二线城市住酒店一天能报销多少钱？；②出差回来后最晚什么时候报销？；③报销差旅费需要什么类型的发票？
    - 在知识库中，将这段“原始文本”和这3个“虚构问题”关联在一起并建立索引。
  - 用户提问阶段：

> 真实提问：“我想报销上周去西安出差的房费，该怎么办？”

  - 匹配过程：
    - 如果直接比对原始文本：①用户说“房费”，文档说“住宿补贴”；②用户说“上周”，文档说“15 个工作日”；③用户说“该怎么办”，文档是陈述句。结果，向量匹配度可能只有 0.6，容易被其他包含“报销”二字的干扰文档带跑。
    - 如果比对“虚构问题”：匹配虚构问题 1，“出差去二线城市住酒店一天能报销多少钱？”（西安是二线城市，语义高度重合）；匹配虚构问题 2，“出差回来后最晚什么时候报销？”（对应用户想报销的动作）。结果，向量匹配度可能高达 0.9，因为提问方式、口吻、关注点完全一致。
  - 生成回答：

> AI 回答：“您去西安（非一线城市）出差，每日房费报销上限是 350 元。请记得在出差结束后 15 个工作日内，通过公司 ERP 系统提交增值税专用发票进行申请哦。”

- 提取标签

- 是一种将“非结构化数据”向“半结构化数据”转化的过程。它通过在语义向量检索的基础上引入标签过滤来**提升检索精度**。
- 例子：
  - 用户搜“iPhone 15 Pro Max”。向量可能会把它和“iPhone 14”放在一起（因为语义很像），但标签匹配能通过`iphone15`这种硬标签精准锚定目标。
  - 效果： 过滤掉大量“语义接近但事实不符”的噪音，解决了“看着像但不是同一个东西”的问题，增加了检索的确定性。

- 重排序（ReRank）

- 检索的优点是可以在海量的知识里面快速找到和用户query相关的内容块docs，但是检索所返回出来的docs，实际上可能部分和用户query关联度并不大，这个时候就需要通过ReRank这一步，对于检索返回出来的docs做关联度排序，最终选取最相关的top k个doc，做后续的上下文补充。
- 原理：在文档的索引阶段，**文档的向量是预先计算并存好的，与Query无关**，这导致它只能表达文档的平均含义。所以，**召回的文档仅能说明与Query相关，但不能保证精确匹配**。*比如，用户提问“推荐一款2000元以下、续航好的华为手机”。Embedding可能会搜回一大堆“华为手机”或“2000元以下手机”，因为这些词的语义权重很大。但它很难平衡这三个条件同时满足的逻辑。*所以，在重排序阶段，利用交叉编码器（Cross-Encoder），把问题（Query）和“候选文档”拼在一起，作为一个长句子塞进模型（比如：gte-rerank）。模型可以同时看到问题和文档的每一个字。通过自注意力机制（Self-Attention），在 Transformer 的每一层中，Query 中的每一个token都可以直接通过 Attention 机制观察到文档中的每一个词。这种交叉注意力（Cross-Attention）能够捕捉到极细微的匹配关系，判定问题与文档的匹配程度。*针对华为手机的例子，强行检查，是不是华为？是不是低于2000？有没有提到续航？如果缺了一项，它的得分会大幅跳水。*
- 重排序优点：
  - 精度飞跃：它是解决“检索回来的资料看着挺像，但就是答非所问”的最佳方案。
  - 多源融合： 如果你同时使用了“向量搜索”和“关键词搜索（BM25）”，两者搜出来的结果排序标准不一样。重排序可以作为统一的裁判，给它们打出公平的分数。
  - 降低大模型负担： 给大模型的内容更少、更准，不仅节省 Token 成本，还能显著提高生成答案的质量。同时能够解决“中间丢失”现象。
- 重排序缺点：
  - 增加延时。
  - 有算力开销。
  - 重排序模型有上下文长度限制。
  - 对查询扩写不友好：如果系统使用了“多查询（Multi-Query）”技术，生成了5个变体问题。那么重排序的计算量会直接乘以 5 倍。

生成答案

- 在检索到相关的文本段后，RAG应用会将问题与文本段通过提示词模板生成最终的提示词，由大模型生成回复，这个阶段更多是利用大模型的总结能力，而不是大模型本身具有的知识。
- 该阶段可能遇到的问题：
  - 没有检索到相关信息，大模型捏造答案。
  - 检索到了相关信息，但是大模型没有按照要求生成答案。
    - 知识冲突，检索回来的 5 个文档块里，文档 A 说“可以办”，文档 B 说“禁止办”。模型陷入困惑。
    - 丢失中间信息。研究表明，LLM 对长上下文的开头和结尾敏感，如果关键答案中间位置，模型很可能视而不见。
    - 模型忽略参考资料。跳过提供的文档，直接用它训练时学到的陈旧知识来回答，导致回答不准确。
- 检索到了相关信息，大模型也给出了答案，但是你希望 AI 给出更全面的答案。
- 优化方式：
  - 选择合适的大模型：
    - 如果只是简单的信息查询总结，小参数量的模型足以满足需求。
    - 如果你希望答疑机器人能完成较为复杂的逻辑推理，建议选择参数量更大、推理能力更强的大模型。
    - 如果你的问题需要查阅大量的文档片段，建议选择上下文长度更大的模型。
    - 如果你构建的 RAG 应用面向一些非通用领域，如法律领域，建议使用面向特定领域训练的模型。
  - 优化prompt：
    - 明确要求不编造答案。一方面，可通过强约束指令，在提示词中明确“禁止”模型使用外部知识；另一方面，可通过强制引用手段，要求模型在句末标注信息来源。
    - 添加内容分隔标记：检索召回的文档切片如果随意混杂在提示词里，人也很难看清整个提示词的结构，大模型也会受到干扰。建议将提示词和检索切片明确地分开，以便大模型能够正确地理解你的意图。
    - 根据问题类型调整模板：不同问题的回答范式可能是不同的，你可以借助大模型识别问题类型，然后映射使用不同的提示词模板。比如有些问题，你希望大模型先输出整体框架，然后再输出细节；有些问题你可能希望大模型言简意赅的给出结论。
  - 模型调参：
    - 如果你希望大模型输出在相同的问题下，输出的内容尽可能相同，你可以在每次模型调用时传入相同的seed值。
    - 如果你希望大模型在回答用户问题时不要总是用重复的句子，你可以适当调高 presence_penalty 值。

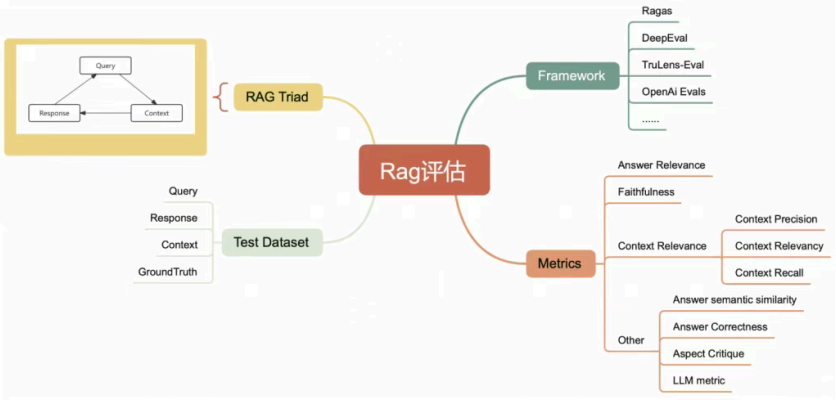
        - 如果你希望查询事实性的内容，可以适当降低temperature 或 top_p 值；反之，查询创造性的内容时，可以适当增加它们的值。
        - 如果你需要限制字数（如生成摘要、关键词）、控制成本或减少响应时间的场景，可以适当降低max_tokens的值，但是若max_tokens过小，可能会导致输出截断，反之，需要生成大段文本时，可以提高它的值。
    - 模型微调：如果提示词优化到了极限，效果仍不理想，就需要考虑微调。
        - SFT（指令微调）： 准备一批“问题 - 参考资料 - 标准答案”的语料，对大模型进行微调。
        - 训练目标： 专门训练模型学会“根据参考资料回答”以及“当资料不足时学会拒绝回答”。

进阶：Graph RAG

- 通过结合知识图谱（Knowledge Graph, KG）和大语言模型（LLM），解决传统 RAG 在处理复杂问题时的局限性。它不仅存储文本向量，还通过 NLP 技术从文本块（trunk）中提取实体（Entity）、关系（Relationship）和属性（Property），构建成一张复杂的知识网（图）。在检索时，可将相连的节点（文本块）一起召回。
- Graph RAG解决的问题：
    - 多跳问题：问题的解决需要多个文本块（trunk）
        - 问题：如果问题是“A的导师的导师是谁？”。文档1 提到了 A 的导师是 B，文档2提到了 B 的导师是 C。
        - 传统 RAG 局限：向量检索可能只找回包含“A 的导师”的片段，由于语义距离，不一定能找回“B 的导师”的片段，导致模型无法回答。
        - Graph RAG 优势：通过图路径（ 文档1 [实体：A、B] <-> 文档2 [实体：B、C] ），可以轻松通过路径追踪找到答案。
    - 全局理解：
        - 问题：这部小说的主旨是什么？
        - 传统 RAG 局限：它擅长“局部搜索”（寻找具体的某段话），但不擅长总结全书，因为它很难一次性把所有文本块都塞进 Prompt。
        - Graph RAG 优势：通过社区检测（Community Detection）算法，可以将图谱划分为不同的层级和簇，对每个簇预先生成摘要，从而实现对全文本的全局概括。
- 工作原理：
    - 第一阶段：索引构建（如何建图）
        - 文本切分：将长文档切成小块。
        - 提取实体与关系：利用 LLM 识别每个文本块中的实体（人、地、事、概念）及其关系（属于、位于、工作于），生成三元组。
        - 图构建：将提取出的三元组汇聚，去重、消歧，构建全局知识图谱。

        - 社区检测（关键步骤，如微软 GraphRAG）：利用图算法（如 Leiden 算法）将紧密相关的实体聚合在一起形成“社区”，并用 LLM 为每个社区生成总结。
    - 第二阶段：查询检索（如何找答案）
        - 局部检索（Local Search）：识别问题中的关键词；在图中定位到具体的实体；提取该实体及其邻居节点（n跳关联）的信息；将这些结构化数据喂给 LLM 生成答案。
        - 全局检索（Global Search）：当问题涉及宏观主题时，不找具体实体；检索预先生成的“社区摘要”。汇总多个社区的结论，给出全局性回答。
- 代表性框架：
    - Microsoft GraphRAG
    - LlamaIndex
    - LightRAG

05 RAG的评估

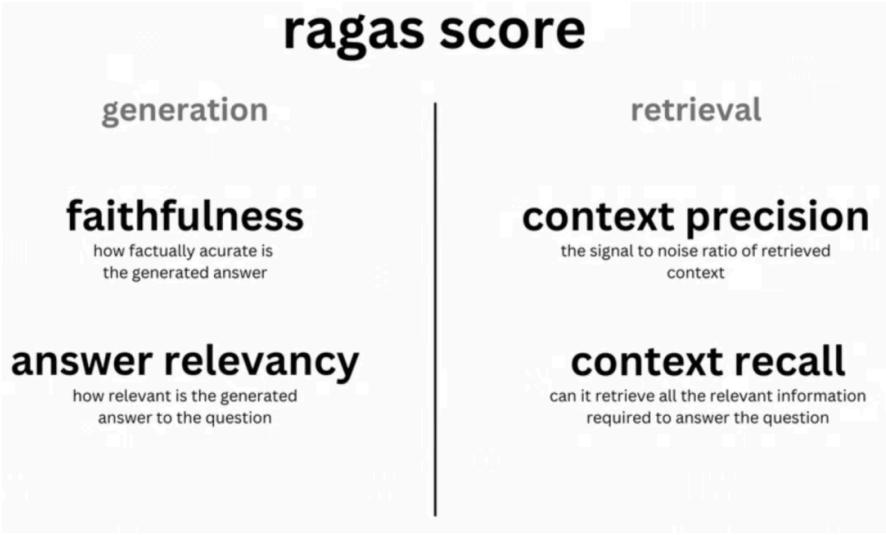


Ragas评估框架

- Ragas（Retrieval Augmented Generation Assessment）是一款广受欢迎的用于 RAG（Retrieval Augmented Generation） 框架效果评估的自动化工具。它提供了上下文（contexts）维度和答案（answer）维度的效果指标计算，实验显示其指标结果的准确性已经非常接近人工评测结果。
- Ragas的核心理念是使用 LLM 来评估LLM（LLM-as-a-Judge），通过一系列自动化的指标来衡量RAG系统的性能。

- 评估指标（Rag Metric）

- Ragas将 RAG 的评估拆解为两个维度，检索（Retrieval）和生成（Generation）。从这两个维度进行评测。



- 检索相关指标：
  - 上下文精度（Context Precision）：评估在针对给定查询（Query）所检索到的上下文中，检索器是否能够将相关的文本块（Chunks）排在比不相关文本块更高的位置。具体而言，**它衡量的是检索出的内容中，相关文本块被放置在排序列表顶部的程度。**
    - 重点：关注检索到的内容对不对；需要标注准确答案。
    - 输入：user_input、reference（参考答案）、retrieved_contexts

$$\text{Context Precision@K} = \frac{\sum_{k=1}^{K} (\text{Precision@k} \times v_k)}{\text{Total number of relevant items in the top } K \text{ results}}$$

$$\text{Precision@k} = \frac{\text{true positives@k}}{(\text{true positives@k} + \text{false positives@k})}$$

  - 上下文召回率（Context Recall）：衡量的是在所有相关的文档中，有多少被成功检索了出来。它的**核心关注点在于“不遗漏”重要结果**。Ragas 中基于 LLM 的上下文召回率指标将“参考答案（Reference Answer）”作为“参考上下文（Reference Contexts）”的替代方案，这使得该指标更易于使用，因为手动标注参考上下文通常极其耗时。**为了根据参考答案估算上下文召回率，参考答案会被拆解为若干个事实点（Claims），随后系统会分析每一个事实点，判断其是否能从检索到的上下文中找到出处**。在理想情况下，参考答案中的所有事实点都应该能够溯源至检索到的上下文。
    - 输入：user_input、reference（参考答案）、retrieved_contexts

$$\text{Context Recall} = \frac{\text{Number of claims in the reference supported by the retrieved context}}{\text{Total number of claims in the reference}}$$

- 生成相关：
  - 忠实度（Faithfulness）：**用于衡量生成的回答与检索到的上下文在事实逻辑上的一致性**。其分值范围为 0 到 1，得分越高说明一致性越好。如果一个回答中所包含的所有陈述点（Claims）都能被检索到的上下文所支撑，则该回答被视为“忠实的”。
    - 输入：user_input，response，retrieved_contexts。

$$\text{Faithfulness Score} = \frac{\text{Number of claims in the response supported by the retrieved context}}{\text{Total number of claims in the response}}$$

  - 答案相关性（Answer Relevancy）：**用于衡量生成的回答与用户输入之间的相关程度**。其分值范围为 0 到 1，得分越高表示回答与用户输入的匹配度越高。该指标的核心在于衡量

回答与问题意图的匹配程度，而不评估事实的准确性。它会对信息不完整或包含冗余细节的回答进行扣分。

- 输入：user_input、response
- 计算方式：①**生成问题**：基于生成的回答，构建一组人工合成的问题（默认为 3 个。可通过LLM构造）。这些问题的设计旨在反映回答中所包含的核心内容；②**计算相似度**：分别计算用户原始输入的embedding向量与每一个生成的合成问题的embedding向量之间的余弦相似度；③**取平均值**： 对这些余弦相似度分数取平均值，从而得到最终的“回答相关性”得分。

$$\text{Answer Relevancy} = \frac{1}{N}\sum_{i=1}^{N}\text{cosine similarity}(E_{g_i}, E_o)$$

$$\text{Answer Relevancy} = \frac{1}{N}\sum_{i=1}^{N}\frac{E_{g_i}\cdot E_o}{\|E_{g_i}\|\|E_o\|}$$

- 噪声敏感度（Noise Sensitivity）：用于衡量系统在利用检索到的文档（无论该文档是相关的还是不相关的）时，**因提供错误回答而产生失误的频率**。该分数范围在 0 到 1 之间，数值越低表示性能越好。
  - 噪声敏感度分类：
    - 相关上下文的噪声敏感度 (Noise Sensitivity - Relevant)：虽然文档相关，但里面包含了一些“参考答案（reference）不需要”的额外信息、背景说明或冗余事实。考察 LLM 能否精准提炼。如果得分高（表现差）：说明模型比较“贪婪”，看到相关的文档就想把里面的所有细节都搬进答案里，即使参考答案没要求。
    - 不相关上下文的噪声敏感度 (Noise Sensitivity - Irrelevant)：检索出来的文档是完全不相关的。这些文档里提到的任何信息对回答当前问题都是干扰。考察LLM能否有效过滤。如果得分高（表现差）：说明模型很容易被“带偏”。它把不相干文档里的东西当成了知识点，编进了答案里。
  - 计算造成敏感度的输入：user_input、reference（参考答案）、response、retrieved_contexts。
  - 计算方式：系统会审查生成回答中的每一个陈述点（Claim），以判断其基于标准答案（Ground Truth）是否正确，以及该陈述是否可以归因于检索到的相关（或不相关）上下文。在理想情况下，回答中的所有陈述都应当由检索到的相关上下文提供支撑。

$$\text{noise sensitivity (relevant)} = \frac{|\text{Total number of incorrect claims in response}|}{|\text{Total number of claims in the response}|}$$

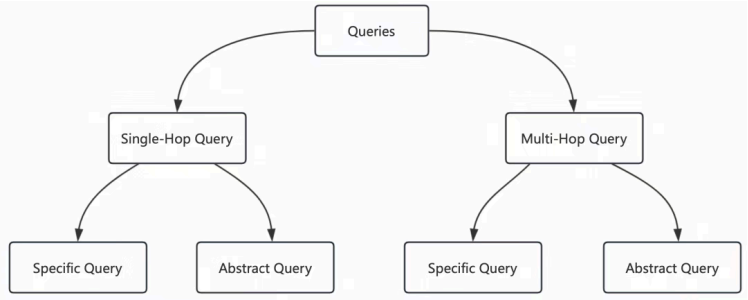
- 评测集生成（Testset Generation for RAG）

- 在实际开发中，开发者往往缺乏高质量的“问题”数据集。Ragas 提供了一个强大的工具，可以自动生成测试集。

如何基于Ragas生成评测集

- 不同模式的查询分类（Query types in RAG）：

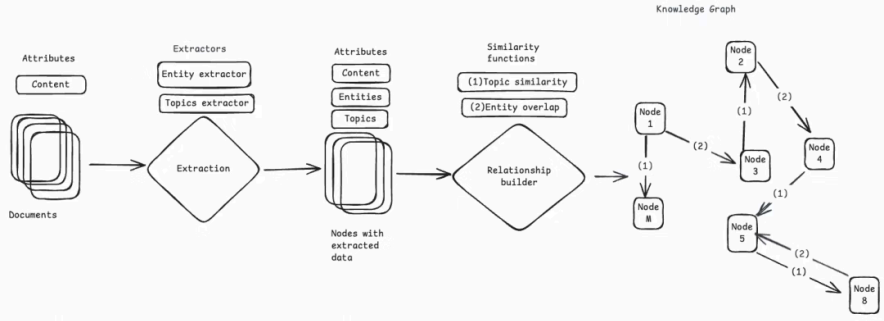
- **单跳查询 (Single-Hop Query)**：仅需从单个文档或来源中检索信息即可提供相关的回答。
  - 具体查询（Specific Query）："阿尔伯特·爱因斯坦是在哪一年发表相对论的？"。这是一个具体的、基于事实的问题，通过从包含该信息的文档中进行一次检索即可回答。
  - 抽象查询（Abstract Query）："爱因斯坦的理论是如何改变我们对时间和空间的理解的？"。虽然该查询仍指向单一概念（相对论），但它需要从源材料中提取出更具抽象性或解释性的说明。
- **多跳查询 (Multi-Hop Query)**：多跳查询涉及多个推理步骤，需要来自两个或更多来源的信息。系统必须从不同的文档中检索信息，并将这些信息点串联起来，才能生成准确的回答。
  - 具体查询（Specific Query）："哪位科学家影响了爱因斯坦关于相对论的研究，他们提出了什么理论？"。这要求系统同时检索出影响爱因斯坦的科学家以及该科学家的具体理论，这些信息可能分布在两个不同的来源中。
  - 抽象查询（Abstract Query）："自爱因斯坦最初发表论文以来，关于相对论的科学理论是如何演变的？"。这种抽象查询需要检索跨越不同时间、不同来源的多条信息，以形成一个关于理论演变的广泛且具解读性的回答。



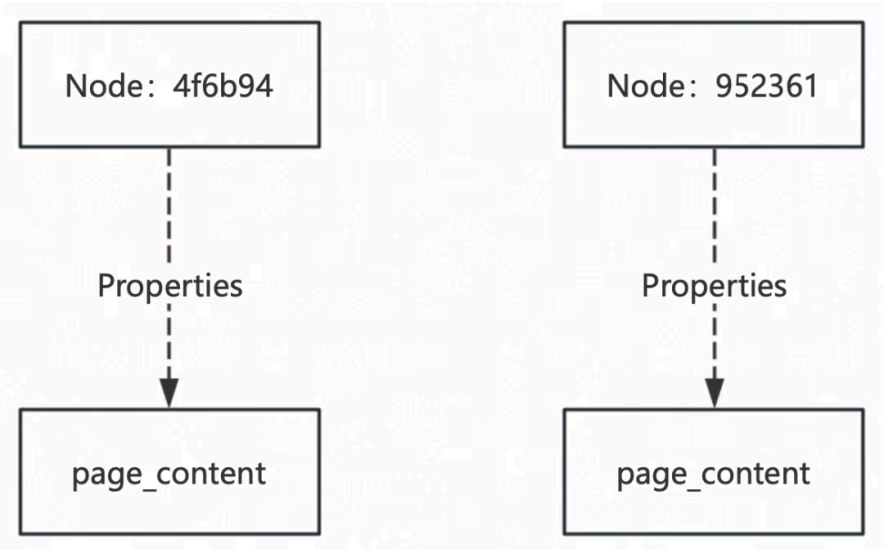
- Specific Query VS Abstract Query：
  - 具体查询 (Specific Query)：侧重于清晰的、基于事实的检索。在 RAG 中，其目标是从一个或多个文档中检索出能直接回答该特定问题的高度相关信息。**侧重于精确性**，*检索出来的知识直接就能用（个人理解）。*
  - 抽象查询 (Abstract Query)：需要更宽泛、更具解释性的回答。在 RAG 中，抽象查询对检索系统提出了挑战，要求其能够从包含高层推理、解释或观点的文档中提取信息，而非简单的客观事实。**侧重于整合广泛的思想**，*需要对检索出来的知识进行抽象总结（个人理解）。*
- Single-Hop Query VS Multi-Hop Query：
  - 单跳查询 (Single-Hop Query)：单条查询的问题生成相对简单。随便找一个文档块，让 LLM 根据这个块提个问题就行。
  - 多跳查询 (Multi-Hop Query)：问题生成困难。如果你想生成一个需要跨越两个知识点的问题（比如：A 影响了 B，B 发明了 C，问：谁影响了 C 的发明者？），你必须

先找到包含 A、B 关系的文档块 1，以及包含 B、C 关系的文档块 2，需要知道各文档块（chunk）间的关系。

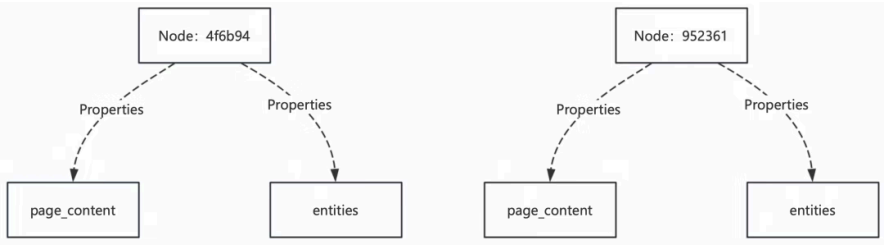
- 知识图谱（Knowledge Graph Creation）：



  - 文档切分（Document Splitter）：



  - 信息提取（Extractors）：系统使用不同的提取器从每个节点中提取信息，这些信息随后被用于构建节点之间的关联。Ragas定义了两类提取器，①基于LLM（大语言模型）的提取器；另一类是基于规则的提取器，比如：NERExtractor。

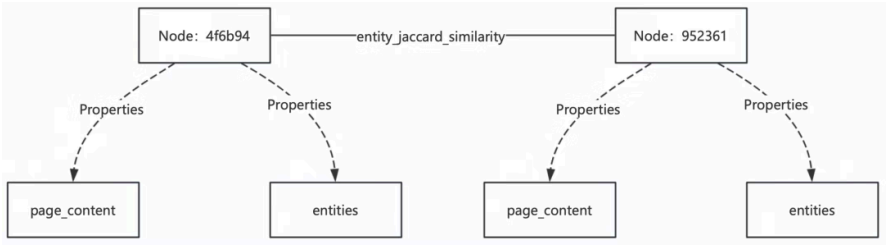


  - 关系构建（Relationship builder）：基于提取出的信息，构建各节点间的关系。支持使用者自定义关系提取器。
    - Ragas内置的Relationship builder：
      a. 基于语义相似度的关联：利用指定的 Embedding 模型将每个 Node 转化为向量，计算其余弦相似度。

b. 基于实体的关联：如果两个 Node 包含相同的实体，Relationship Builder 就会在它们之间建立关联。

c. 基于层级/结构的关联：在文档切分时记录各trunk间的关系（相邻段落、主子关系）。通过层级/结构构建关系。

d. 基于关键词/关键短语的关联：利用 Keyphrase Extractor 提取节点中的高频词或核心短语。适合于在没有明确命名实体的情况下（比如纯技术文档或理论说明），通过共同的关键术语建立连接。



- 场景生成（Scenario Generation）：知识图谱构建完成后，可以使用他来构造任何类型的问题。当不同背景的用户群体与 RAG 系统交互时，他们表达查询的方式会大不相同，这取决于他们的人设(Persona)、查询长度以及查询风格。为了生成能够覆盖所有这些真实情况的问题，Ragas 在测试集生成中采用了一种基于场景 (Scenario-based) 的方法。
  - 测试集生成中的每个“场景”都是以下参数的组合：

a. 节点 (Nodes)：用于生成该查询的特定知识图谱节点。

b. 查询长度 (Query Length)：期望生成的查询长度，可以是短、中、长等。

c. 查询风格 (Query Style)：查询的表达风格，可以是网络搜索式（Web search）、聊天对话式（Chat）等。

d. 人设 (Persona)：提出问题的用户角色，例如：高级工程师、初级工程师等。

  - 实操：可通过内置的Query Synthesizer或自定义Query Synthesizer来完成场景生成以及评测集的产出。

```python
from dataclasses import dataclass
from ragas.testset.synthesizers.base_query import QuerySynthesizer
@dataclass
class EntityQuerySynthesizer(QuerySynthesizer):
    async def _generate_scenarios( self, n, knowledge_graph, callbacks
        """
        这个方法不调用大模型去写问题，它只是在查地图（KG）。
          1. 输入：你想要几道题（n），以及知识图谱（KG）。
          2. 它做的事：
            在 KG 中寻找有关系的节点。
            决定这n道题分别用哪些节点作为素材。
            给每道题贴上标签（如：这道题用“正式”风格，由“经理”身份提问）。
          3. 输出：一组 Scenario（方案列表）。
        """
        return scenarios
```

```
    async def _generate_sample(
        self, scenario, callbacks
    ):
        """
        1. 输入：一个 Scenario（包含节点文本、人设、风格等）。
        2. 它做的事：
          把文本和要求塞进 Prompt。
          调用 LLM。
          让 LLM 返回：问题是什么？标准答案是什么？。
        3. 输出：一个 TestsetSample（一道完整的题目）。
        """
        return SingleTurnSample(user_input=query, reference_contexs=co
```

06 参考文档

- Ragas：https://docs.ragas.io/en/latest/getstarted/
- 阿里云ACP课程：https://github.com/AlibabaCloudDocs/aliyun_acp_learning/tree/main/%E5%A4%A7%E6%A8%A1%E5%9E%8BACP%E8%AE%A4%E8%AF%81%E6%95%99%E7%A8%8B

07 团队介绍

本文作者驰海，来自淘天集团-天猫品牌行业架构团队。团队聚焦构建面向品牌行业的定制化AI Agent架构能力。我们致力于构建面向行业的AI Agent全链路架构基础设施，涵盖选型、复用、评测与自进化闭环。围绕品牌经营核心场景，团队深度赋能商品运营全链路智能托管、数字员工驱动的智能客服体系升级以及AIGC素材生成与内容生产等方向，通过提供可规模化、高质量的Agent架构解决方案，持续驱动AI技术在品牌行业的深度应用与价值释放。

¤ 拓展阅读 ¤

3DXR技术 | 终端技术 | 音视频技术

服务端技术 | 技术质量 | 数据算法

大淘宝技术
大淘宝技术官方账号
967篇原创内容
公众号

阅读 2.4万

留言

写留言

还没有留言，来写首评